MTH 4300
Spring 2026

Programming Problem Set 6: Operator Overloading - `vector3d.cpp`

Directions: Download the template files provided on Brightspace. Submit your source code via Gradescope, in `.cpp` format.

Submission Reminder: Submitting for feedback is optional. If you submit by the deadline and want feedback beyond the autograder, indicate that in your submission. You may resubmit unlimited times.
Important: Passing all test cases does not guarantee correctness. The autograder tests limited inputs. Test additional cases yourself.
Note: While learning, avoid IDE tools that autocomplete or generate code for you (e.g., Copilot, IntelliSense suggestions). Writing code yourself builds the understanding you'll need for exams.

Working with AI on This Assignment

This assignment has a structured AI component. Follow these steps in order.

Step 1: Write your code on your own first. Give each part a genuine attempt before using any AI. If your code compiles and runs but gives wrong answers, try tracing through it with a specific input on paper before moving on to Step 2.

Step 2: If you're stuck, use AI as a debugging tutor. Don't paste your code and ask "what's wrong." Instead, use a prompt like this:

> *"I'm writing a C++ class called Vector3D that [describe what it should do]. Here is my code. I'm getting [wrong output / crash / compiler error]. Don't fix my code or show me corrected code. Instead, pick a specific line and ask me what I think a particular variable or expression evaluates to at that moment. Help me find the bug by making me trace through my own logic. Keep a back-and-forth conversation."*

If you use this step, save the conversation; you'll submit a link to it.

Step 3: After your code passes the autograder, do a code review conversation. The goal is to test whether you can explain what you wrote. Paste one of the prompts below along with your working code into a new AI conversation (ChatGPT, Claude, Copilot, etc.). The AI will quiz you; treat it like a study session. Operator overloading involves several design decisions (member vs. non-member functions, `const` correctness, return types). Walking through your choices out loud is a good way to solidify your understanding.

Paste the prompt below along with your working code (vector3d.cpp) into a new AI conversation:

> *"Here is my working C++ code for a Vector3D class (vector3d.cpp). Act as a technical interviewer. Ask me questions one at a time about my design and implementation choices. Start easier and build toward harder questions. If my answer is vague or wrong, don't correct me — ask a follow-up that helps me see the issue. Do 5–6 rounds. Then propose a small modification to my class and ask me to implement it. Critique my implementation."*

Submit a link to the conversation as a comment at the top of your `.cpp` file, along with any Step 2 conversation links.

Be sure to read the SPECIFICATIONS carefully! And write comments!

General Specifications and Notes:

- No global variables should be used.
- The only header allowed is `iostream`.
- You cannot use vectors or any advanced data structure that can store an arbitrarily long collection of numbers.
- Note: there is no user input.
- Remember to use `const` whenever appropriate.

Note: Each problem has test cases on Gradescope. All test cases are visible, so you can see exactly which cases your code passes or fails. Use this feedback to improve your solution. You may resubmit as many times as you like. Remember, these assignments are for practice; use them to prepare for exams!

Note that when we use the term *vector* below, we mean it in the mathematical sense of the word, not as in the C++ data structure.

6a) **Arithmetic and comparison operators**

Write a C++ class called `Vector3D` to represent 3D vectors according to the following specifications:

- The class should have three private attributes of type `double`, representing an x-coordinate, y-coordinate, and z-coordinate.
- A constructor that takes three arguments for setting the attributes and, by default, sets them all to 0.
- Three accessor methods `getX()`, `getY()`, and `getZ()` that return the respective coordinates.
- Overload the + and `-`; i.e., addition and subtraction operators so that addition and subtraction of 3D vectors occur coordinate-wise. For example $(1, 2, 3) + (4, 5, -3) = (5, 7, 0)$; $(1, 2, 3) - (4, 5, -3) = (-3, -3, 6)$.
- Overload the $*$ operator so it corresponds to scalar multiplication on the right. For example, $(1, 2, 3) * 2 = (2, 4, 6)$.
- Overload the $*$ operator to be used as a dot product between two 3D vectors. For example, $(1, 2, 3) * (4, 5, 6) = 1 * 4 + 2 * 5 + 3 * 6 = 32$.
- Overload the == operator for vectors so it evaluates to true when all components of the vectors match.

The template file includes some tester code with what should be printed. Include only the class definition in your final submission and no main function.

6b) **Friend functions, compound assignment, and more**

Add the following to your `Vector3D` class:

- Overload the $*$ operator using a `friend` function so that scalar multiplication also works on the left. For example, $2 * (1, 2, 3) = (2, 4, 6)$. Combined with part (a), both $(1, 2, 3) * 2$ and $2 * (1, 2, 3)$ should give the same result.
- Overload the += and `-=` compound assignment operators so that they modify the left-hand vector in place. For example, if $v_1 = (1, 2, 3)$ and $v_2 = (4, 5, 6)$, then after `v1 += v2`, v_1 should be $(5, 7, 9)$.
- Write a method `cross()` that takes another `Vector3D` as an argument and returns the cross product. Recall that the cross product of (a_1, a_2, a_3) and (b_1, b_2, b_3) is $(a_2b_3 - a_3b_2,\ a_3b_1 - a_1b_3,\ a_1b_2 - a_2b_1)$.
- Overload the `< <` operator so that when an object of the `Vector3D` class is put into a `cout` statement, it is displayed as three numbers, separated by commas, with enclosing parentheses (and no newline at the end). There should be spaces after the commas. For example, like `(1, 2, 3)`.
- Overload the `> >` operator so that when an object of the `Vector3D` class is put into a `cin` statement, then the three doubles entered are used to set the attributes of the object.

You will submit this part in a separate submission area on Gradescope (the same `vector3d.cpp` file, now with the above additions included).

6c) **AI Code Review + Practice Questions**

AI Code Review: After your code passes the autograder, complete the AI code review conversation described at the beginning of this assignment. Submit a link to the conversation as a comment at the top of your `.cpp` file. See the "Working with AI on This Assignment" box on page 1 for the prompt and full instructions.

The following are extra practice questions for you. They are good preparation for exams. Try them on your own, and let me know if you have questions.

1. Suppose you wanted to add a `magnitude()` method that returns the length of the vector (i.e., $\sqrt{x^2 + y^2 + z^2}$). Where would you add it, and what would the return type and signature look like? Would you need any additional headers?
2. Suppose you wanted to overload `!=` in addition to ==. Write the implementation. Could you implement it in terms of ==? If so, how?

3. Explain the difference between implementing the `+` operator as a member function versus as a non-member friend function. Which did you choose, and why? What changes would be needed to switch to the other approach?
4. Could you overload `*` to compute the cross product between two `Vector3D` objects, in addition to the dot product? Why or why not?

MTH 4300 S26